

Heart Disease Detection using Explainable AI

Demo / Viva Q&A Study Guide

Logistic Regression · Random Forest · SVM
Explained with SHAP and LIME · Validated with McNemar's test, calibration, fairness checks

This guide has one job: make sure you can answer, in your own words, any question your professor or HOD asks about this project tomorrow. Every answer below is short enough to say out loud in 15-30 seconds, uses plain language first, and only brings in a technical term after the plain-language version. Read it once tonight, then skim it again 30 minutes before your demo.

How to use this guide: don't try to memorise word-for-word. Read each answer, then close your eyes and say it back in your own words. If you can explain the idea simply, you understand it - and that's what actually gets marks.

Quick-Reference Cheat Sheet

Memorise this page above everything else. If you remember nothing but these numbers, you can survive almost any question.

Dataset	70,000 rows -> cleaned to 68,052 (1,948 removed, 2.78%)
Train / Test split	54,441 train / 13,611 test (80/20, stratified)
Features used	11 (age_years, gender, bmi, ap_hi, ap_lo, pulse_pressure, cholesterol, gluc, smoke, alco, active)
Logistic Regression	Accuracy 73.1% ROC-AUC 0.790 best C = 0.01
Random Forest	Accuracy 73.4% ROC-AUC 0.798 best model overall
SVM (RBF)	Accuracy 73.2% ROC-AUC 0.792 trained on 15,000-row sample
Model agreement	Average gap between any two models: 4.7-5.5 points (target was <10)
McNemar's test	All 3 models: $p > 0.05$ -> NOT statistically different from each other
Calibration method	CalibratedClassifierCV, isotonic regression, 5-fold
Explainability tools	SHAP (game theory, exact) + LIME (local approximation)
Top 3 risk features	Systolic BP (ap_hi), Age, Cholesterol
Website pages	8: Dashboard, Prediction, SHAP, LIME, Batch CSV, Model Performance, Clinical Validation, About
Notebooks	11, numbered 01 to 11, run in order

1. General Project Questions

Q: What is this project?

It predicts whether a patient has cardiovascular disease using 11 simple health measurements - things a nurse can record in a few minutes, like blood pressure, age, cholesterol and weight. Three different machine learning models make the prediction, and then two explainability tools, SHAP and LIME, show exactly why each model decided what it decided.

Q: Why does the project use three models instead of one?

Because if three completely different types of algorithm - a straight line, a forest of decision trees, and a curved boundary - all point to the same conclusion, that conclusion is much more trustworthy than any single model's opinion. We even proved this formally later with a statistical test.

Q: Why does explainability matter for a medical project?

A hospital will never accept an AI that just says '80% risk' and refuses to explain itself. A doctor needs to know why, so they can judge whether the reasoning makes medical sense before trusting the number. That's the whole reason SHAP and LIME exist in this project - they open the black box.

Q: What dataset did you use?

The Cardiovascular Disease dataset from Kaggle, by Svetlana Ulianova - 70,000 real patient examination records with 11 input features and one target (has heart disease or not).

NUMBERS TO MENTION: 70,000 raw rows, 68,052 after cleaning

Q: How long did this project take you?

About three and a half weeks, working through it step by step - understanding the data first, then cleaning, then exploring it, then training and tuning three models, then building the explainability pages, and finally the validation and the website.

Q: What tools and libraries did you use?

Python, with pandas and numpy for data handling, scikit-learn for the three models, SHAP and LIME for explainability, Streamlit for the website, and Plotly for the interactive charts.

Q: Is this a medical device? Can it actually diagnose someone?

No, and I want to be clear about that. This is an academic screening tool, not a diagnostic device. It hasn't been clinically validated or approved for real use. What it's good for is ranking patients by risk so a doctor's attention goes to the highest-risk cases first - the final decision always stays with a human doctor.

2. Data Understanding and Cleaning

Q: What columns are in the raw dataset?

Thirteen columns: id, age (in days), gender, height, weight, two blood pressure numbers (ap_hi and ap_lo), cholesterol, glucose, smoking, alcohol, physical activity, and the target column cardio - 0 for healthy, 1 for heart disease.

Q: Was the raw data clean?

No - and finding that out was an important step. The raw file had blood pressure readings as high as 16,020 mmHg, which is medically impossible. It also had negative diastolic readings and people recorded as 55 cm tall. These are data-entry errors, not real patients.

NUMBERS TO MENTION: Max ap_hi in raw data: 16,020 mmHg (a real human max is around 200)

Q: How did you clean the data? What rules did you use?

I removed duplicate patients, kept only blood pressure readings between medically sensible ranges (systolic 90-200, diastolic 60-130), required the top BP number to actually be bigger than the bottom one, kept height between 140-200cm and weight between 40-150kg, and finally filtered BMI to between 15 and 50.

NUMBERS TO MENTION: 1,948 rows removed out of 70,000 - only 2.78%

Q: Why did you only remove 2.78% of the data? Isn't that too little?

That's actually the right amount. Removing too little would leave impossible values in the training data; removing too much would mean throwing away real patients just because their numbers looked unusual. I checked that the healthy/disease balance stayed the same before and after cleaning, which proved the cleaning was fair and didn't accidentally delete mostly one group.

Q: Were there any missing values?

No - the dataset had zero missing values in every column. The problem wasn't missing data, it was impossible data.

Q: Was the target balanced? Why does that matter?

Yes, almost exactly 50/50 - 49.5% of patients have heart disease. That matters because if it were, say, 95% healthy, a lazy model could just always guess 'healthy' and still score 95% accuracy without learning anything. Because it's balanced, accuracy is a meaningful score here, and I didn't need any special balancing technique like SMOTE.

NUMBERS TO MENTION: 49.46% of patients have heart disease

3. Exploratory Data Analysis (EDA)

Q: What did you find during EDA? Which features matter most?

Three features stood out clearly, just from plotting and counting, before any model was even trained: systolic blood pressure, age, and cholesterol. Disease rate rises steadily with age - about a quarter of patients have it at age 30-40, but roughly two out of three do by age 60-70.

Q: Why did you do EDA before training any model?

Two reasons. First, to understand the data properly before trusting it to a model. Second - and this was actually important later - it gave me an independent baseline. When SHAP later told me which features the models relied on, I could check it against what I'd already found by simple counting. They matched, which proved the whole pipeline was consistent.

Q: You mentioned smoking and alcohol looked harmless in your data - isn't that wrong?

It looked surprising, yes, but there's a real explanation, not a bug. Smoking, alcohol, and activity were self-reported - the patient just told the doctor, and people commonly under-report smoking and drinking. Blood pressure and cholesterol were actually measured by a nurse or a lab, so they're far more reliable. This is a finding about data quality, not a mistake in my code.

Q: What is the correlation heatmap for?

It shows which measurements move together, and specifically which ones move together with heart disease. The bottom row of the heatmap - correlation with the target - is basically a preview of what the machine learning models will later find important.

4. Feature Engineering

Q: What is feature engineering, and what did you create?

It means building new, smarter columns out of the existing raw ones. I created three: `age_years` (the raw file stored age in days, which nobody thinks in), BMI (weight divided by height squared - the number doctors actually use instead of raw weight), and `pulse_pressure` (top BP minus bottom BP, which doctors read as a sign of stiff arteries).

Q: Why not just use raw age, height and weight directly?

Because that would be redundant with the engineered features. BMI already combines height and weight into one meaningful number - feeding in all three would split the credit between them in SHAP and make each look less important than it really is. Also, age in days (like 18,393) has no medical meaning to a human reading the explanation.

Q: Why did you split the data before scaling?

This is one of the most important rules in machine learning: the model must be judged on questions it has never seen, like a sealed exam paper. I locked away 20% of patients before doing anything else, so the final test score is honest.

NUMBERS TO MENTION: 80/20 split, stratified - 54,441 train, 13,611 test

Q: What is 'data leakage' and how did you avoid it?

Data leakage is when information from the test set accidentally influences training, making your score look better than it really is. I avoided it very specifically: the `StandardScaler` was fit only on the training data, then just applied to the test data. If I'd fit it on all the data together, the test set's own statistics would have leaked backwards into training.

Q: Why do Logistic Regression and SVM need scaled features but Random Forest doesn't care as much?

Logistic Regression and SVM both work by measuring distances or gradients between numbers. Blood pressure runs up to 200 while smoking is just 0 or 1 - without scaling, blood pressure would completely dominate just because its numbers are bigger, not because it's more important. Random Forest splits on one feature at a time using thresholds, so the scale of the numbers matters much less to it - but I scaled everything the same way anyway, for consistency across all three models.

5. The Three Models

Q: Explain Logistic Regression in simple words.

Imagine plotting every patient on a graph. Logistic Regression draws one straight line and says everyone on one side is probably healthy, everyone on the other side probably has heart disease. It then converts how far a patient is from that line into a probability.

NUMBERS TO MENTION: Accuracy 73.1%, ROC-AUC 0.790, best setting $C=0.01$

Q: Explain Random Forest in simple words.

A single decision tree asks a chain of yes/no questions, like 'is blood pressure above 140? Then is age above 55?' One tree alone can be fooled by noise, so a Random Forest grows 200 different trees, each trained on a slightly different random slice of the data, and lets them vote. The crowd is wiser than any one tree.

NUMBERS TO MENTION: Accuracy 73.4%, ROC-AUC 0.798 - the best of the three

Q: Explain SVM in simple words.

SVM looks for the boundary with the widest possible safety margin between healthy and sick patients. The RBF kernel lets that boundary curve instead of being a straight line, by imagining the patients lifted into a higher-dimensional space where a curve becomes flat.

NUMBERS TO MENTION: Accuracy 73.2%, ROC-AUC 0.792, trained on a 15,000-patient sample

Q: Why did you train the SVM on only 15,000 patients instead of the full 54,441?

Because an RBF SVM's training time grows roughly with the square of the number of patients - training on all 54,000 would take hours. I used a random, class-balanced sample of 15,000 instead. This is a standard, defensible engineering trade-off, and the accuracy came out essentially identical to the other two models anyway, which proves the sample was representative.

IF THEY PUSH FURTHER: If they ask 'how do you know the sample was fair', say: I checked the disease rate in the sample matched the full training set almost exactly.

Q: What is GridSearchCV and why did you use it?

It automatically tries several different settings for a model and keeps whichever one scores best, using cross-validation to judge fairly. I used it so I never hand-picked a setting and hoped for the best - every choice is backed by evidence.

Q: What is cross-validation and why does it matter?

It splits the training data into several parts (I used 5), trains the model multiple times leaving out a different part each time, and checks the score is close every time. If all the scores are close together, the result is real and stable, not a lucky one-time split.

Q: What hyperparameters did you tune for each model?

For Logistic Regression: C , which controls how strict versus flexible the decision boundary is. For Random Forest: `max_depth` (how deep each tree can grow) and `min_samples_leaf` (the minimum patients allowed at the end of a branch). For SVM: C and γ , which control the margin width and how far each patient's influence reaches.

6. Calibration and Comparing the Models

Q: What is probability calibration and why did you use it?

A model's raw output is a score, not necessarily an honest probability - it might say 80% for a group where only 60% are really sick. Isotonic calibration re-maps the scores so that among patients told '70%', roughly 70% really are sick. I needed this because my website shows all three models' percentages side by side, and without calibration they wouldn't be comparable to each other.

NUMBERS TO MENTION: Method: CalibratedClassifierCV, isotonic regression

Q: How closely do your three models agree with each other?

Very closely - the average gap between any two models is under 6 percentage points, comfortably inside the 10-point target I set for the project.

NUMBERS TO MENTION: LR vs RF: 5.2pp | LR vs SVM: 5.5pp | RF vs SVM: 4.7pp | average: 7.7pp overall

Q: Which model is the best?

Random Forest scores slightly highest on most metrics. But here's the more important answer: I ran a statistical test, McNemar's test, and it showed the difference between all three models is not statistically significant. So while Random Forest wins on paper, I wouldn't confidently claim it's truly better than the other two.

IF THEY PUSH FURTHER: This is your strongest answer in the whole project. Say it confidently - it shows you understand statistics, not just coding.

Q: What is McNemar's test and why did you use it?

It's the standard statistical test for comparing two classifiers on the exact same test patients. It ignores the cases both models agree on and only looks at where they disagree, then checks if one model is disagreeing in its favour significantly more often than the other.

NUMBERS TO MENTION: Every pairwise p-value came back above 0.05, e.g. LR vs RF: $p=0.227$

Q: If the models are statistically the same, why not just use one?

That's actually the right conclusion, and it's a mature one - when models tie statistically, the right way to choose is by interpretability, not by chasing the third decimal place of accuracy. I'd deploy Logistic Regression in a real hospital because its coefficients are directly readable by a doctor.

Q: Is 73% accuracy good? Why not higher?

It's exactly right for this dataset. Published research on this same Kaggle dataset sits between 72% and 74% - my numbers match. There's no ECG, no family history, and cholesterol is only a rough 1-2-3 category rather than a real lab number, so there's a genuine ceiling on what 11 basic measurements can predict. Anyone reporting 95% on this exact dataset has almost certainly leaked test data into training.

IF THEY PUSH FURTHER: If pushed further: 'I'd rather report the honest 73% I can defend than an inflated number I can't explain.'

Q: Is 50% the right cut-off point to decide 'sick' vs 'healthy'?

No, and I checked this specifically. A missed sick patient is far more dangerous than a false alarm - one goes home undiagnosed, the other just gets an extra check-up. I modelled a missed patient as three times worse than a false alarm and found the cutoff that minimises total harm. It's much lower than 50%, and it catches far more genuinely sick patients at the cost of more false alarms.

NUMBERS TO MENTION: Cost-optimal cutoff for Random Forest: 27%, catching ~1,500 more sick patients

7. SHAP Explainability

Q: What is SHAP, in simple words?

SHAP comes from game theory - specifically, how do you fairly split a prize among a team that won it together? SHAP treats each health measurement as a player and the final prediction as the prize. It starts from the average patient, then adds up how much each measurement of your specific patient pushes the prediction up or down.

Q: What makes SHAP special compared to other explanation methods?

It has a mathematical guarantee: the contributions always add up exactly to the model's real prediction. I actually demonstrate this in my notebook - I show the starting point plus all the SHAP values equals precisely what the model output.

NUMBERS TO MENTION: Example: base 54.1% + SHAP values = exactly the model's real prediction

Q: What is the difference between local and global SHAP?

Local SHAP explains one specific patient - why did the model give this person 80%? Global SHAP averages that effect across many patients, to show what the model relies on in general. I use both: local on the Prediction and SHAP pages for one patient, global to build the feature-importance chart.

Q: What did SHAP find was most important?

The same three features EDA found by simple counting: systolic blood pressure, age, and cholesterol. All three of my models agreed on this too, even though they use completely different mathematics.

IF THEY PUSH FURTHER: Mention this cross-check unprompted if you get the chance - it's genuinely impressive and most projects don't do it.

Q: What is the SHAP background set, and why only 20 patients?

SHAP needs a 'typical patient' baseline to compare against. Using all 54,000 training patients would take hours, so I summarised them into 20 representative patients using k-means clustering - this keeps the maths honest while making it fast enough for a live website.

8. LIME Explainability

Q: What is LIME, in simple words?

LIME stands for Local Interpretable Model-agnostic Explanations. The idea: the real model might be very complicated, but if you zoom in close to just one patient, its behaviour looks almost like a simple straight line. LIME creates thousands of slightly-changed copies of your patient, asks the real model about each one, then fits a simple, readable model to those answers - giving readable rules like 'systolic BP above 140 means risk up'.

Q: Why use both SHAP and LIME? Isn't one enough?

Because an explanation you can't check is just another black box. SHAP and LIME use completely different mathematics - one is game theory, the other is a local approximation. When two independent methods agree on the same top features, the explanation is real, not an accident of one particular technique.

Q: Why does LIME's percentage sometimes differ slightly from the real model's answer?

Because LIME's percentage comes from its simple copy, not from the real model directly. A small gap is completely normal - the R-squared score tells you how good the copy is. What matters is the ranking and direction of the rules, not LIME's own percentage.

Q: Why did you tell LIME which features are 'categorical'?

Without that, LIME would treat cholesterol level 3 as if it were a point on a continuous scale and write nonsense rules like 'cholesterol greater than 1.5'. By marking gender, cholesterol, glucose, smoking, alcohol and activity as categories, LIME writes sensible rules like 'cholesterol = 3' instead.

9. Clinical Validation (Beyond Accuracy)

Q: What is on your Clinical Validation page, and why did you build it?

Most student projects stop at accuracy. I wanted to answer questions a real hospital review board would actually ask: is the best model really better, or just lucky? Is 50% the right decision line? Can the probabilities be trusted at face value? Would more data help? Does the model treat every patient group fairly?

Q: What is a Brier score and what does it tell you?

It's a single number measuring how well-calibrated a model's probabilities are - lower is better, zero is perfect. All three of my models score close together and close to zero, confirming that when the app says '70% risk', that number can be trusted, not just used as a rough ranking.

NUMBERS TO MENTION: Brier scores: LR 0.186, RF 0.182, SVM 0.184 - all close together

Q: What are learning curves and what did they show you?

A learning curve trains the model on increasingly larger slices of data and tracks training accuracy versus validation accuracy. If the two lines are still pulling apart at the end, more data would help. Mine converge and flatten, which means the ~73% ceiling comes from the data itself, not from under-training - more patients wouldn't meaningfully improve the score.

Q: What is the subgroup fairness check?

A model can look accurate overall while quietly performing worse for one specific group - for example, catching fewer sick women than sick men. I checked accuracy and recall separately across gender and age bands to look for exactly this kind of hidden imbalance before calling the model deployment-ready.

Q: What is 'out-of-distribution' detection, and why did you add it?

The models were only ever trained on patients with BMI between 15 and 50 - anything outside that was removed during cleaning as a data-entry error. I actually tested my own sliders at their extremes and found that a patient with an impossible BMI made the three models disagree by over 20 points. So I added a live warning: if you build a patient outside that trained range, the app tells you the prediction is an extrapolation and should be treated with extra caution.

IF THEY PUSH FURTHER: This is a great answer if they ask 'what did you find while testing your own project' - it shows real engineering, not just following a tutorial.

10. The Website

Q: What pages does your website have?

Eight: a Dashboard with data exploration, Patient Prediction showing all three models at once, a SHAP page, a LIME page, Batch CSV prediction for scoring many patients at once, Model Performance with all the metrics, Clinical Validation for the deeper statistical checks, and an About page explaining the whole project.

Q: What technology is the website built with?

Streamlit, a Python framework that turns a script into an interactive web app without needing separate frontend code. Charts are built with Plotly so they're interactive - you can hover, zoom, and pan.

Q: Can the models be retrained from scratch? How do I prove that to my professor?

Yes - the entire pipeline is reproducible. If you delete everything inside the models folder and rerun the 11 notebooks in order, every model, every metric file, and even the Clinical Validation data regenerate themselves from nothing but the raw CSV file. I tested this myself in a completely empty folder to make sure it actually works, not just assumed it would.

Q: Why 11 separate notebooks instead of one big file?

Each notebook does one clear job in the pipeline - understand the data, clean it, explore it, engineer features, train each of the three models separately, compare them, explain them with SHAP then LIME, and finally assemble everything. Splitting it up means anyone reading it, including a first-year student, can follow exactly what happens at each stage without getting lost in one giant file.

Q: Does your website support batch predictions on many patients at once?

Yes - you can upload a CSV file of many patients, and all three models score every row at once. It includes a downloadable template, automatically detects if you upload the raw Kaggle-style file and converts it, flags rows outside the trained BMI range, and lets you download the full results.

11. Tricky Questions They Might Throw At You

Q: If I change one slider slightly, why does the prediction jump around?

It shouldn't jump wildly for small changes - the models are smooth functions. If it does jump a lot, it's usually because the patient has crossed a genuine medical threshold, like blood pressure crossing 140, which is the real hypertension cutoff doctors use. That's actually the model correctly picking up on a real medical rule.

Q: What would you do differently if you had more time?

Four things: add richer clinical features like ECG readings or family history; try a gradient-boosting model like XGBoost as a fourth comparison point; tune the decision threshold specifically for recall rather than accuracy, since missing a sick patient is worse than a false alarm; and validate the model on data from a different hospital to check it generalises beyond this one dataset.

Q: Did you write all this code yourself?

I worked through this project step by step over about three and a half weeks, using AI tools like Claude to help me write, test and understand the code as I went - the same way a developer uses documentation, Stack Overflow, or a senior colleague. What matters is that I understand what every part does and why it's there, which is exactly what this guide is helping me practice.

IF THEY PUSH FURTHER: Be honest if asked directly. Own the understanding, not just the output - explaining WHY a choice was made is what actually proves you understand it.

Q: What was the hardest part of this project?

Making sure the data was actually clean before trusting it to any model - finding impossible values like a 16,020 blood pressure reading, and deciding exactly which medical rules to use to filter them out without accidentally deleting real patients.

Q: What's the difference between accuracy and ROC-AUC?

Accuracy is simply how many of 100 patients you labelled correctly. ROC-AUC asks a different question: if you picked one sick patient and one healthy patient at random, how often does the model correctly give the sick one a higher risk score? ROC-AUC is more robust because it doesn't depend on picking exactly the right 50% cutoff.

Q: Why didn't you use a neural network or deep learning?

With only 11 simple features and 68,000 rows, a deep neural network would add complexity without adding real predictive power - there just isn't enough hidden pattern in the data for deep learning to find that classical models can't already find. It would also make the explainability story much harder, since deep networks are notoriously difficult to interpret compared to the models I chose.

12. Final Reminders Before Your Demo

- Speak slowly and confidently. A short correct answer beats a long nervous one.
- If you don't know something exactly, say what you DO know and reason out loud - professors respect thinking over silence.
- Your strongest answer is the McNemar's test result - the models are statistically tied, so you chose interpretability. Say this with confidence.
- Your second strongest answer is the dirty data (16,020 mmHg blood pressure) - it shows you actually looked at your data instead of blindly trusting it.
- If your partner freezes on a question, you can politely help by starting the answer and letting them finish it - that's normal teamwork, not cheating.
- Never say 'the AI did it' - say 'I built this by working through the data step by step' and then explain the actual reasoning. You clearly understand it - trust that.

Good luck tomorrow. You've put real work into this project - now go explain it like you own it, because you do.